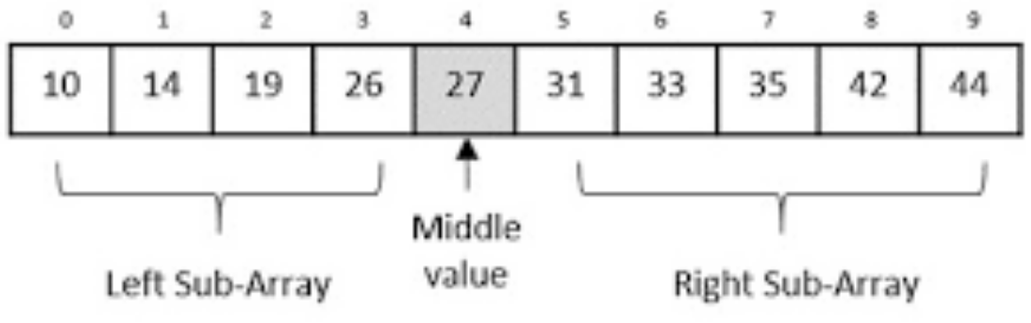
Binary Search and Variants

- Applicable whenever it is possible to reduce the search space by half using one query

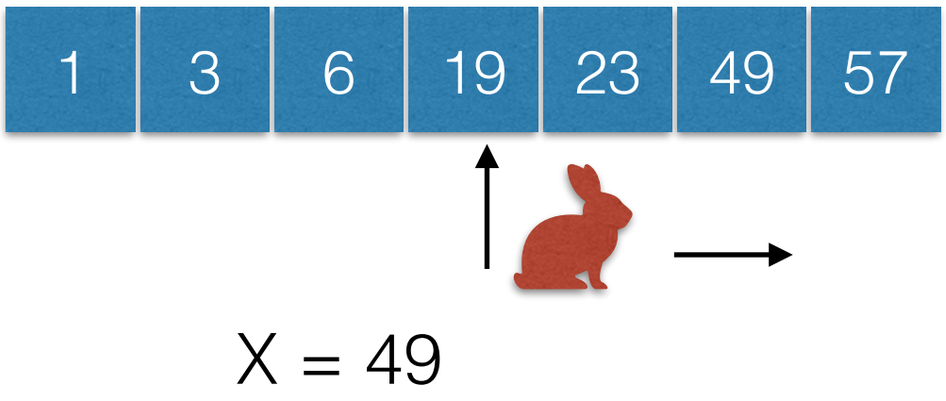
- Search space size N
  number of queries = O(*log* N)

- Ubiquitous in algorithm design. Almost every complex enough algorithm will have binary search somewhere.



Classic example

- Given a sorted array A of integers,
  find the location of a target number x
  (or say that it is not present)



Pseudocode:

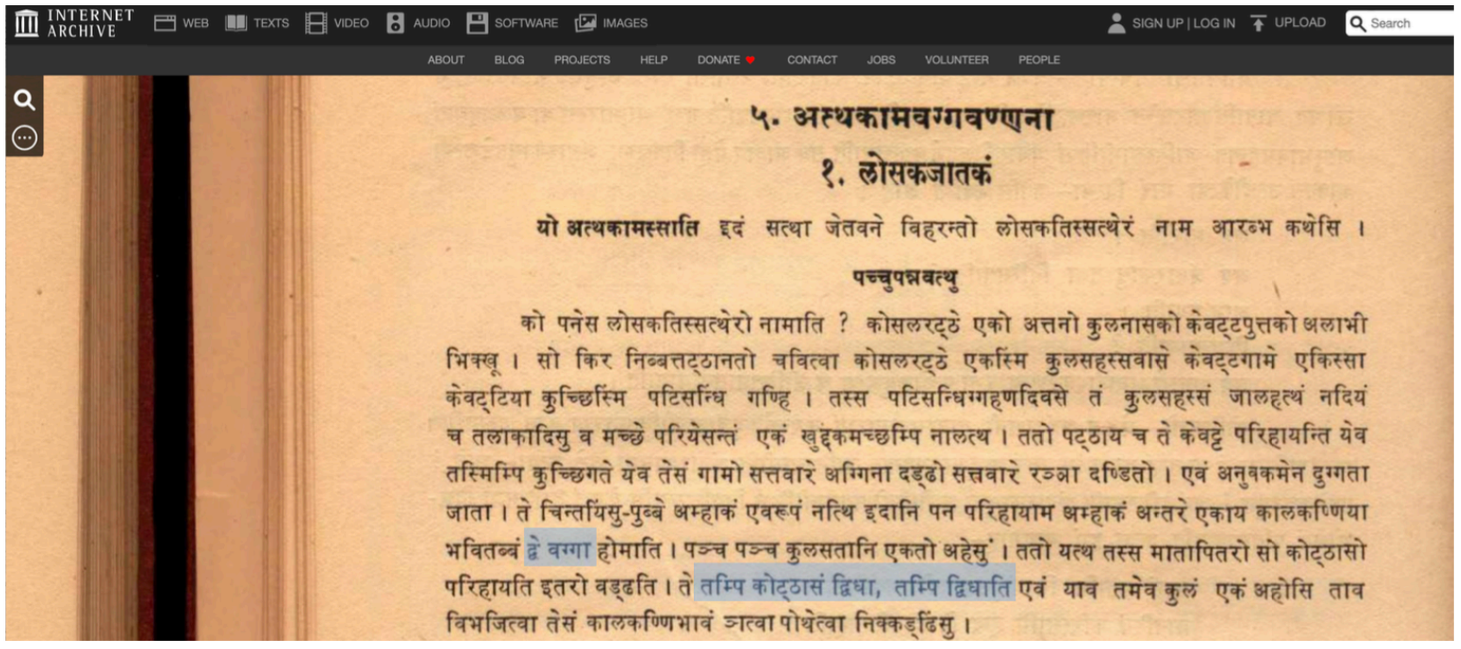
```
Initialize start ← 0, end ← n;
Locate(x, start, end){
    if (end < start) return not found;
    mid ← (start+end)/2;
    if (A[mid] = x) return mid;
    if (A[mid] < x) return Locate(x, mid+1, end);
    if (A[mid] > x) return Locate(x, start, mid-1);
}
```

History

- Binary search was first first mentioned by John Mauchly (1946) - The art of computer programming, vol 3, p 422.

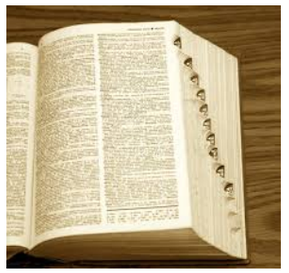
- Later popularised by Donald Knuth.

- Mention in ancient texts, e.g., Buddhist Jātaka Tales…

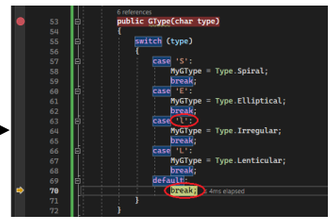


Other examples

- Looking for a word in the dictionary →



- Debugging a linear piece of code →

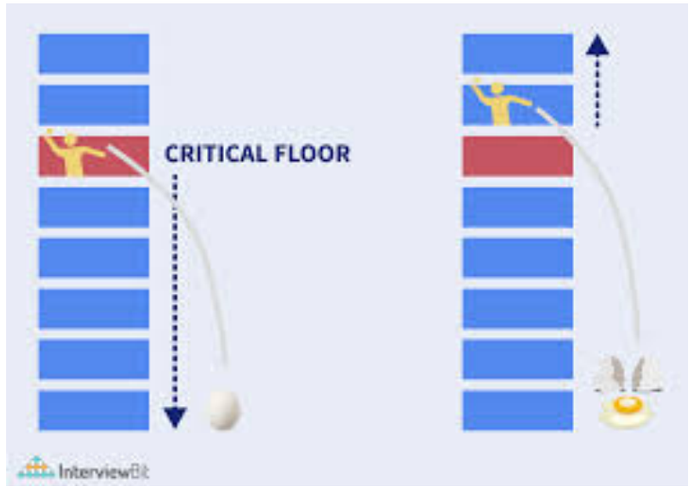


- Cooking rice →

- Science/Engineering: Finding the right value of any resource
  - length of youtube ads
  - pricing of a service

Egg drop problem

- In a building with n floors,
  find the highest floor from where egg can be dropped without breaking.
- $O(\log n)$ egg drops are sufficient.
- Binary search for the answer.
- Drop an egg from floor x
  - if the egg breaks, answer is less than x
  - if the egg doesn't break, the answer is at least x
- Using the standard binary search idea, start with $x=n/2$.
  If egg breaks, then go to $x=n/4$ and it doesn't then go to $x=3n/4$.
  And so on



Egg drop: unknown range

- In a building with *infinite* floors,
  find the highest floor *h* from where egg can be dropped without breaking.
- *O(log h)* egg drops sufficient?
- Exponential search: Try floors, 1, 2, 4, …, till the egg breaks.

Egg drop: unknown range

- The egg will beak at floor 2^{k+1}, where $2^k \leq h < 2^{k+1}$
- Then binary search in the range $[2^k , 2^{k+1}]$
- Total number of egg drops $\leq k+1+\mathrm{k} = 2k+1 \leq 2\ log\ h +1$.
- Is there a better way?

Lower bound

- Search space size: N
  Are $\log N$ queries necessary?

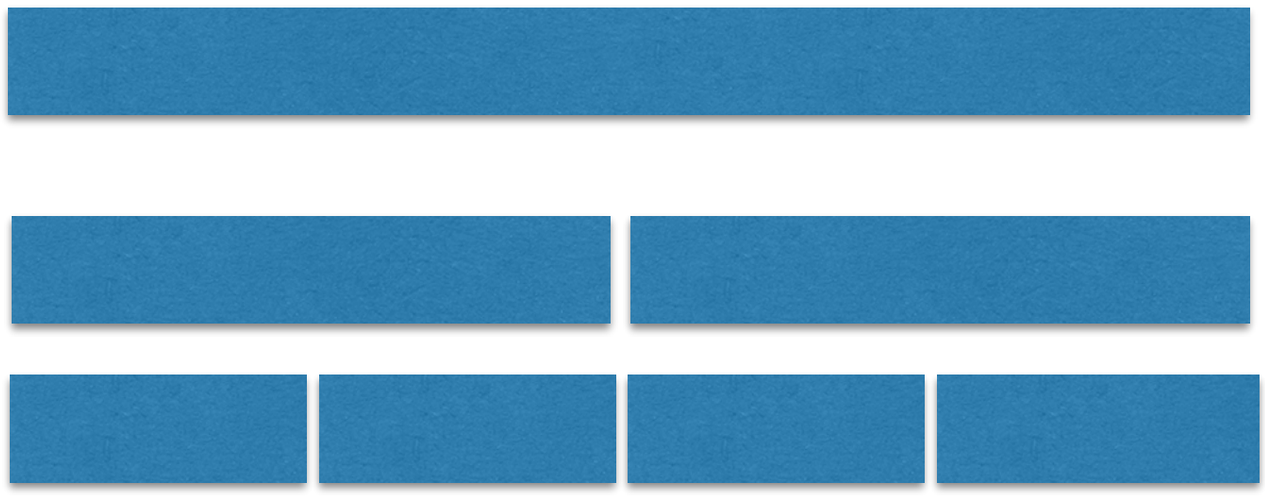
- Yes. When each query is a yes/no type, then the search space gets divided into two parts with each query
  (some solutions correspond to yes and others to no).

- One of the parts will be at least $N/2$.

Lower bound

- In worst case, with each query, we get the larger of the two parts.

- To reduce the search space size to 1, we need *log N* queries



Lower bound

- Search space size: N
  Are $log\ N$ queries necessary?

- Another argument based on Information Theory.

- Each yes/no query gives us 1 bit of information.

- The final answer is a number between 1 and N, and thus, requires $log\ N$ bits of information.

- Hence, $log\ N$ queries are necessary.

- Ignore this argument if it is hard to digest.

Exercise: subarray sum

- Given an array with n positive integers,
  and a number S,
  find the minimum length subarray whose sum is at least S?

- Subarray is a contiguous subset, i.e.,
  A[i], A[i+1], A[i+2], …, A[j-1], A[j]

  - [10, 12, 4, 9, 3, 7, 14, 8, 2, 11, 6]

    S = 27

- Can we do this in *O(n log n) time*?

Exercise: subarray sum

O(n^2) algorithm:

```
for (l ← 1 to n){
        T ← sum of first l numbers.
        for (j ← 0 to n-l-1){
            if T ≥ S return l and the subarray (j, j+l-1);
            T ← T - A[j] + A[l+j];
        }
}
```

- Two for loops one inside the other. Each makes at most n iterations. Hence, O(n^2) time.

Exercise: subarray sum

O($n \log n$) algorithm. Approach 1:

Binary search for the minimum length l.

For the current value of l:

- check if there is a subarray of length l with sum at least S.
This check can be done in $O(n)$ time. See the inner loop on previous slide.

Exercise: subarray sum

- O(*n log n*) algorithm. Approach 2:
- First compute all the prefix sums and store in an array. O(n) time.

- prefix_sum[0] ← A[0];
  for (i ← 1 to n-1)
      prefix_sum[*i*] = prefix_sum[*i-1*]+A[*i*];

- Any subarray sum from *i* to *j* can now be computed in O(1) time as prefix_sum[*j*] - prefix_sum[*i-1*].
- Now, for each choice of starting point, do a binary search for the minimum end point such that the subarray sum is at least *S*.
- If sum of a subarray < *S*, then choose a larger end point, otherwise smaller.

Exercises

- Given two sorted arrays of size n, find the median of the union of the two arrays.
  $O(\log n)$ time?

- Given a convex function $f(x)$ oracle, find an integer x which minimizes $f(x)$.

- Land redistribution:
  given list of landholdings $a_1, a_2, \ldots, a_n$,
  given a floor value f,
  find the right ceiling value c

Division algorithm

- As we find the next digit of the quotient, the search space of the quotient goes down by a factor of 10.

- This could be called a denary search.

- For binary representation of numbers, the division algorithm will be a binary search.

Finding square root

- Given an integer a, find $\sqrt{a}$
- Start with a guess x
- If $x^2 > a$, then the answer is less than x
- Else the answer is at least x.
- This way we can do binary search for $\sqrt{a}$
- Additional exercise: find a division like algorithm.

Finding square root

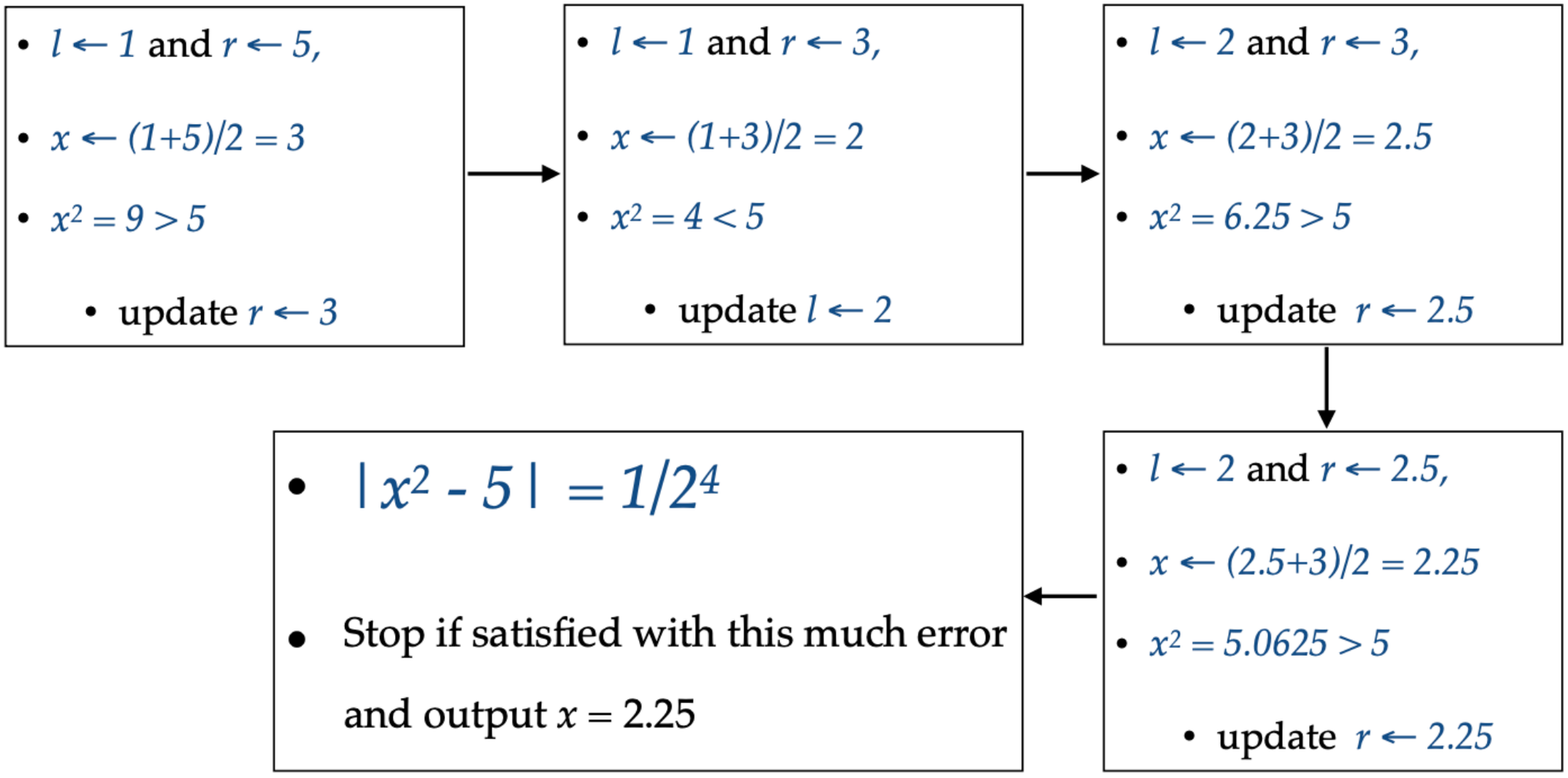
- Given an integer a, find $\sqrt{a}$ up to l digits after decimal.

- Can we compute it in O(l) iterations?

- Yes.

Finding square root (Example)

- Computing *sqrt(5)*.



More applications

- Finding inverse of an increasing function $f(x)$
- Finding root of an odd-degree polynomial?
- Finding the smallest prime dividing N ?
- Is sorting a kind of binary search?
  $O(n \log n)$ comparisons necessary?

More applications

- Finding inverse of an increasing function?

- For any given x, we have a method to compute $f(x)$.
  For a given y, we want to compute $f^{-1}(y)$.

- Make a guess x and compare it $f(x)$ with y

- If $f(x) < y$ then the answer is larger than x

- Else the answer is at least x.

More applications

- Finding a root of polynomial $f(x)$?
- Always maintain two points a and b such that $f(a) > 0$ and $f(b) < 0$.
- To find the starting points one can do exponential search.
- Check whether $f((a+b)/2) > 0$.
- If yes, then there is a root between $(a+b)/2$ and b.
- Else, there is a root between $(a+b)/2$ and a.

More applications

- Finding the smallest prime dividing N ?
- No.

- We can make a guess x. If x does not divide N, then we cannot say anything about where should be the smallest prime dividing N.

More applications

- Is sorting a kind of binary search?
  $n \log n$ comparisons necessary?
  - Yes.
- When we have not made any comparisons, then any of the $n!$ rearrangements is a possible answer.
  - So the search space size is $n!$.
  - Each comparison will reduce the search space size by only 1/2 (in worst case).
- Hence, $\log (n!) \geq n \log n - n \log e$ comparisons are necessary.